

JAVA Stream API Practice Questions

LEVEL 1: BASICS (Stream creation, filtering, mapping, counting)

Goal: Understand basic Stream operations, predicates, and transformations.

1. Stream creation & printing

- Given a list of integers, print each element using `forEach()`.
- Create a stream from an array and print only even numbers.

2. Filtering

- Print all strings that start with a given letter.
- From a list of numbers, filter only those > 10 .
- Given a list of names, filter those with length > 5 .

3. Mapping

- Convert a list of strings to uppercase.
- Convert a list of integers to their squares.
- Given names, extract their first character into a list.

4. Counting & matching

- Count how many numbers are greater than 100.
- Check if all elements are even.
- Check if any string contains "java".

5. Collecting results

- Collect all even numbers into a list.
- Convert a stream of integers to a `Set`.
- Join strings with a comma using `Collectors.joining()`.

LEVEL 2: INTERMEDIATE (flatMap, sorting, distinct, statistics)

Goal: Learn stream chaining, flattening, and aggregations.

6. Sorting & distinct

- Sort a list of numbers in descending order.
- Remove duplicates from a list of strings.
- Sort employee names alphabetically.

7. flatMap – flattening streams

- Given a list of lists of integers, create one flat list.
- From a list of sentences, extract all words as a single stream.
- Given a list of customers with orders, list all product names.

8. Aggregation & statistics

- Find the max and min of a list using `max()` and `min()`.
- Compute average salary of employees.
- Sum all integers greater than 50.
- Find second-highest number using streams.

9. Grouping & partitioning

- Group words by their length.
- Group employees by department.
- Partition numbers into even and odd.

10. Collectors & summaries

- Get a summary of integer list (sum, average, min, max).
- Convert a list to a `Map<String, Integer>` where key=name and value=length.
- Convert a stream to `Map<Department, List<Employee>>`.

LEVEL 3: FUNCTIONAL THINKING (composing operations)

Goal: Combine filters, maps, and collectors logically.

11. Conditional chaining

- Given list of products, get names of those with price > 100 and category = "Electronics".
- From a list of students, get names of top 3 scorers with score > 80.

12. Complex transformations

- From a list of sentences, count total number of unique words.
- Convert list of names into `map<firstLetter, List<String>>`.
- Given a list of transactions, find total spending per user.

13. Working with Optional & Stream

- Safely handle a stream that might be empty using `Optional`.
- Find first number greater than 50 or return default.
- Convert `Optional` to `Stream` and process further.

14. Chaining example

- Given list of employees, find the name of the highest-paid in each department.
 - From a list of movies, find titles of top 3 highest-rated in 2024.
-

● LEVEL 4: ADVANCED (custom collectors, parallel streams, performance)

Goal: Learn complex aggregation, reduction, and optimization.

15. Reduction (`reduce()`)

- Compute factorial using `reduce`.
- Find the longest string in a list.
- Concatenate all strings using `reduce`.

16. Custom collectors

- Create a collector to join names with “|”.
- Build a collector to compute average length of strings.
- Implement a custom collector that collects integers into a comma-separated String.

17. Parallel streams

- Use parallel stream to sum 1–1,000,000 numbers.
- Compare performance of parallel vs sequential for large datasets.
- Discuss when **not** to use parallel streams (thread-safety cases).

18. Advanced grouping & downstream collectors

- Group employees by department, then by role.
- Compute total salary per department.
- Get department-wise highest paid employee.

19. Nested stream processing

- Given `List<Company>` → each has multiple `Department` → each has employees:
 - Print all employee names across all companies.
 - Find highest salary across all companies.
 - Group employees by company name.

● LEVEL 5: EXPERT (stream design, debugging, custom pipelines)

Goal: Solve real-world, multi-step pipeline problems.

20. Full data processing pipelines

- Given a CSV of students (name, grade, score):
 - Filter those with grade = "A".
 - Compute average score per grade.
 - Get top scorer's name per grade.

21. Stream API with enums & nested data

- Given a list of orders with status (NEW, SHIPPED, DELIVERED):
 - Count how many are pending.
 - Group orders by status.
 - Find average price per status.

22. Stream with dates and times

- From a list of `LocalDate`, find all dates in current month.
- Sort by most recent date.
- Group dates by year.

23. Debugging and optimization

- Use `peek()` for debugging transformations.
- Avoid multiple traversals (show example with performance gain).
- Compare `.map().filter()` vs `.filter().map()` in performance.

24. Real-world data modeling

- Given a list of employees:
 - Find employees earning in the top 10%.
 - Get department with maximum average salary.
 - Find employee names sorted by joining year then salary.

25. Mixed data types & flat processing

- Given nested JSON-like data (e.g., `List<Map<String, Object>>`):
 - Flatten all values that are integers.
 - Collect all keys into a `Set`.



BONUS CHALLENGES

To reach complete fluency

26. **Implement custom Stream-like behavior** (build your own `SimpleStream` wrapper).
 27. **Lazy evaluation test:** Show that `filter` and `map` are lazily executed.
 28. **Short-circuiting test:** Demonstrate difference between `findFirst`, `findAny`, and `limit`.
 29. **Combine Streams of different types:** Merge `Stream<String>` and `Stream<Integer>` intelligently.
 30. **Infinite Stream:** Use `Stream.iterate()` to generate Fibonacci sequence and limit first 10 terms.
-



PROGRESSION PLAN (Recommended Order)

Level	Topics	# of Problems	Key Concepts
1	Basics	15	Stream creation, filter, map, collect
2	Intermediate	15	flatMap, grouping, distinct, sort

Level	Topics	# of Problems	Key Concepts
3	Functional Thinking	10	chaining, optionals, pipelines
4	Advanced	10	reduce, custom collector, parallel
5	Expert	10	multi-step real- world pipelines